```
  const [state, dispatch] = React.useReducer(
    venueReducer,
    initialState(props)
  )
  React.useEffect(() => {
    const fetchData = async () => {
      const response = await fetch(
        `/tickets.json?concert_id=${props.concertId}`
      )
      const tickets = await response.json()
      dispatch({ type: "setTickets", tickets: tickets })
    }
    fetchData()
  }, [])

  return (
    <VenueContext.Provider value={{ state, dispatch }}>
      <Venue />
    </VenueContext.Provider>
  )
}

export default App
```

The file starts with some declarations. We start by importing React, then importing our `Venue` component and a few names from new files called `venue_context` and `venue_types` that we'll talk about in a moment. Then we declare two TypeScript interfaces, one for the `AppProps` and one that will be used for the context called `IsVenueContext`. The `AppProps` interface describes the data we just passed to this component as props: the concert ID, the number of rows, and the number of seats in a row.

The `IsVenueContext` has two elements: a `state` element of type `VenueState`, which is defined in the context file, and a `dispatch` element of type `React.Dispatch<VenueAction>`. Could this be related to the state and dispatch we just talked about for reducers and stores? Oh, the suspense…

The context is created next, outside the actual component function:

```
export const VenueContext = React.createContext<IsVenueContext>(null)
```